

Rapport Projet SESI

Shifter et Multiplieur MIPS Vectoriel

Auteurs :

Tifenn FABRICI

Asmar MOHAMED

Louis DAI

Encadrant :

Pirouz BAZARGAN-SABET

Master SESI
Année 2025-2026

Contents

1 Mise en Contexte **2**
1.1 Shifter (Décalage) . . . 2
1.1.1 Les différents types d'opérations de décalage . . . 2
1.1.2 L'Architecture en "Barrel Shifter" . . . 2
1.2 Multiplieur . . . 2
1.2.1 Les différentes opérations . . . 4
1.3 MIPS Vectoriel . . . 4

2 Problématique **4**

3 Tâches **4**
3.1 Shifter paramétrable . . . 4
3.1.1 Schéma du Shifter paramétrable . . . 4
3.2 Multiplieur paramétrable . . . 6
3.2.1 Schéma du Multiplieur paramétrable . . . 6
3.2.2 Formattage et découpage des différentes opérandes . . . 6
3.3 Comparaison des Multiplieurs sur FPGA . . . 9

4 Procédure de recette **10**
4.1 Tests pour le Shifter et le Multiplieur . . . 10

1 Mise en Contexte

Notre projet se définit par une approche à double dimension : un aspect théorique dédié à la conception et à l'optimisation des schémas d'architecture, et un aspect applicatif à travers l'implémentation et la validation des codes VHDL.

Les opérations de décalage et de multiplication sont des éléments centraux de l'unité d'arithmétique et logique (ALU) d'un processeur.

1.1 Shifter (Décalage)

L'unité de décalage (Shifter) est un composant critique du processeur, dont l'architecture traite **quatre opérations** fondamentales : le **décalage à droite signés** et **non signés**, le **décalage gauche**, ainsi que les **rotations** à **gauche** et à **droite**.

1.1.1 Les différents types d'opérations de décalage

On distingue généralement trois types de manipulations au sein d'une unité de décalage universelle :

- **Décalage Logique (SLL/SRL) :** Les bits sont décalés et les positions libérées sont systématiquement remplies par des zéros. C'est l'opération de base pour multiplier ou diviser un nombre non-signé par une puissance de deux.
- **Décalage Arithmétique (SRA) :** Lors d'un décalage à droite, le bit de poids fort (MSB) est dupliqué pour préserver le signe du nombre. Cela permet de conserver la cohérence mathématique des nombres signés en complément à deux.
- **Rotation (ROL/ROR) :** Aucun bit n'est perdu; les bits sortants d'un côté sont réinjectés de l'autre, créant un décalage circulaire de la donnée.

1.1.2 L'Architecture en "Barrel Shifter"

Pour réaliser ces opérations de manière performante, l'architecture la plus répandue est le *Barrel Shifter*. Contrairement à un registre à décalage classique qui nécessiterait plusieurs cycles d'horloge, le *Barrel Shifter* est un circuit combinatoire capable de réaliser un décalage de n bits en un seul cycle.

Cette performance est obtenue grâce à une structure en **étages logarithmiques**. Pour un mot de N bits, le shifter est composé de $\log_2(N) + 1$ étages. Chaque étage i est piloté par un bit de commande b_i et effectue (ou non) un décalage de 2^i positions. En cascade, ces étages permettent de couvrir toutes les amplitudes de décalage possibles (de 0 à $N - 1$).

Dans une architecture 8 bits le Shifter ressemblera à :



Figure 1: Architecture du Shifter 8 bits et détails des composants

1.2 Multiplieur

Multiplier deux nombres est une opération fréquente dans la plupart des programmes.

L'opérateur de multiplication se repose sur le même principe que la multiplication posée apprise à l'école, sauf que la base 2 est utilisée au lieu de la base 10.

$$
\begin{array}{rcccccccc}
 & & & A_3 & A_2 & A_1 & A_0 & & \\
 & \times & & B_3 & B_2 & B_1 & B_0 & & \\
\hline
 & & A_3B_0 & A_2B_0 & A_1B_0 & A_0B_0 & & & \\
 & A_3B_1 & A_2B_1 & A_1B_1 & A_0B_1 & & & & \\
A_3B_2 & A_2B_2 & A_1B_2 & A_0B_2 & & & & & \\
+ \quad A_3B_3 & A_2B_3 & A_1B_3 & A_0B_3 & & & & & \\
\hline
P_7 & P_6 & P_5 & P_4 & P_3 & P_2 & P_1 & P_0
\end{array}
$$

Figure 2: Multiplication posée sur 4 bits

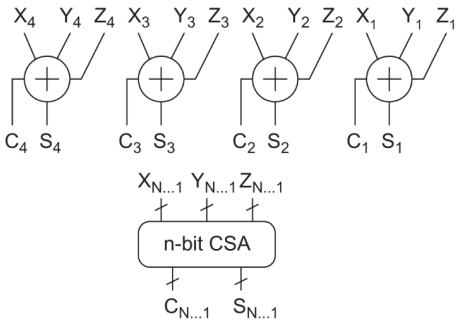


Figure 3: Carry Save Adder

En revanche, pour des raisons évidentes de performance, les processeurs modernes souhaitent éviter de sommer les produits partiels générés. Pour cela, nous utilisons ce que l'on appelle un *Arbre de Wallace* qui prend un ensemble de produits partiels et les additionnes.
C'est une architecture se reposant sur l'utilisation d'un *Compresseur* 3:2 nommé "Carry Save Adder" (CSA) pour grandement accélérer le calcul et qui prend 3 éléments en entrée pour en ressortir 2: une somme bit à bit modulo 2 et la retenue de chaque colonne d'une addition.
À noter qu'il est néccessaire de sommer les deux sorties du CSA pour obtenir la "vraie" somme des éléments d'entrés.

En combinant plusieurs couches de CSA et un additionneur final, nous obtenons cet arbre dont l'avantage est son délai logarithmique.

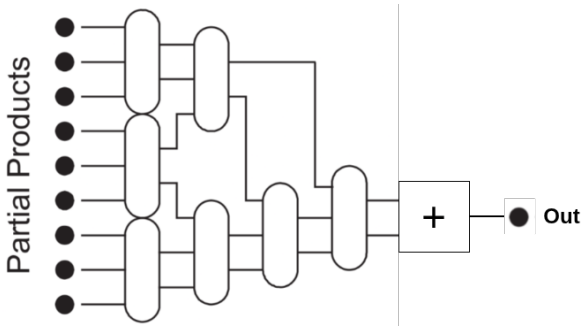


Figure 4: Arbre de Wallace

Le Multiplieur Scalaire utilisé dans notre processeur MIPS se repose sur cette architecture et est de plus capable d'additionner ou soustraire une opérande Z, ainsi que de gérer des opérandes signées.

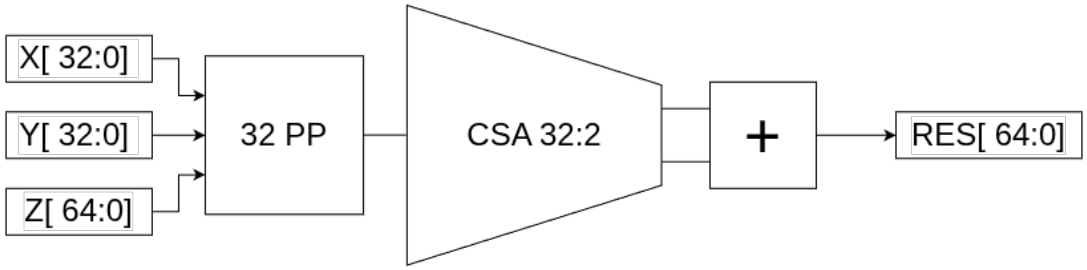


Figure 5: Schéma du Multiplieur Scalaire 32 bits

1.2.1 Les différentes opérations

Mul-Add

- *Non signé*

$$R = Z + X \cdot Y$$

- *Signé*

$$R = \begin{cases} Z + X \cdot Y & \text{si } Y \geq 0 \\ \overline{\overline{Z} + X \cdot \overline{Y} + X} & \text{si } Y < 0 \end{cases}$$

Mul-Sub

- *Non signé*

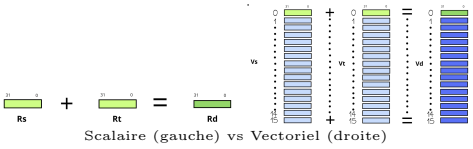
$$R = Z - X \cdot Y = \overline{\overline{Z} + X \cdot Y}$$

- *Signé*

$$R = \begin{cases} \overline{\overline{Z} + X \cdot Y} & \text{si } Y \geq 0 \\ Z + X \cdot \overline{Y} + X & \text{si } Y < 0 \end{cases}$$

1.3 MIPS Vectoriel

L'Architecture utilisée est un MIPS32 **avec un vecteur de 16 mots**. Ainsi, les instructions de cette architecture seront différentes d'un MIPS Scalaire. Voici la différence visuelle dans le cas d'une addition :



Scalaire (gauche) vs Vectoriel (droite)

2 Problématique

Dans le cadre du traitement de signal ou de l'apprentissage statistique, les opérations de multiplication et de décalages sont souvent utilisées pour réaliser des produits matriciels. Or une architecture classique pour effectuer ces opérations engendrerait des problèmes de performances. Paralléliser ces calculs devient alors une nécessité.
Ce projet a pour but de concevoir et vérifier une version "Single Instruction Multiple Data" (SIMD) de ces opérateurs pour un processeur MIPS32 Vectoriel.

Plus précisément, l'opérateur possède 3 configurations: Word/Half/Byte
Cela lui permet donc de traiter au plus 4 vecteurs en parallèle.

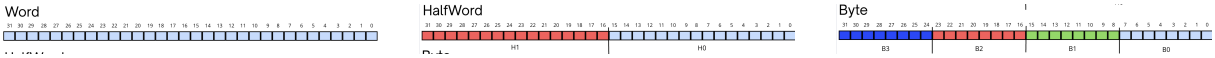


Figure 6: Configuration des opérandes

Tout l'enjeu est donc de concevoir des architectures capables de basculer entre ces trois modes et d'effectuer son calcul en un seul cycle d'horloge. Cela en mutualisant les portes logiques pour minimiser la surface et le délai.

3 Tâches

3.1 Shifter paramétrable

3.1.1 Schéma du Shifter paramétrable

Le shifter paramétrable 32 bits constitue une extension directe de la logique 8 bits, permettant d'effectuer des décalages adaptés à différents formats : **bytes**, **halfs** ou **words**.

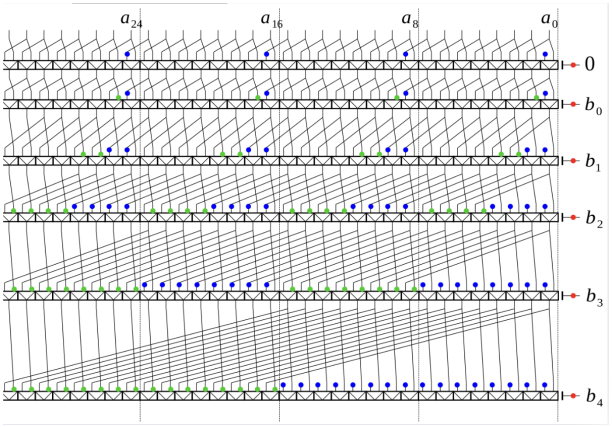


Figure 7: Architecture globale du Barrel Shifter 32 bits paramétrable

Pour construire cette architecture, nous avons répété la structure d'un shifter 8 bits sur les quatre premiers étages. Cette répétition permet de traiter en parallèle les quatre octets du mot de 32 bits pour les décalages fins :

- **Étage 0** : Gère l'activation globale ou le passage direct de la donnée.
- **Étage** b_0 : Effectue un décalage élémentaire de 1 bit.
- **Étage** b_1 : Effectue un décalage de 2 bits.
- **Étage** b_2 : Effectue un décalage de 4 bits.

À chaque étage, des **multiplexeurs** (boules bleues) agissent comme des commutateurs : ils choisissent soit de laisser passer le bit tel quel, soit de le dérouter vers une position voisine décalée. Afin de ne pas surcharger visuellement le schéma, les connexions relatives aux boules bleues n'ont pas été tracées.

Afin de couvrir la largeur totale du mot de donnée et de rendre l'unité paramétrable, deux étages ont été intégrés :

- **Étage** b_3 : Spécifiquement ajusté pour gérer les **halfs** (demi-mots), il permet un décalage de 8 bits. Ici, les multiplexeurs connectent les fils d'un octet vers l'octet adjacent.
- **Étage** b_4 : Il finalise l'unité en permettant un décalage de 16 bits. À ce niveau, les multiplexeurs réalisent un croisement à grande échelle en acheminant les données d'un demi-mot (16 bits) vers l'autre.

Cette organisation permet d'obtenir n'importe quel décalage ou rotation, de 0 à 31 bits, en combinant l'activation de ces différents étages de manière séquentielle selon le type de donnée traité.

3.2 Multiplieur paramétrable

3.2.1 Schéma du Multiplieur paramétrable

Pour réaliser notre Multiplieur à partir du Multiplieur Scalaire, il suffit de rajouter un formattage des opérandes pour la formation des produits partiels et ainsi qu'un second formattage de la somme en sortie de l'arbre de Wallace, en fonction du mode Word, Half, Byte et du signe.
Le détail de l'implémentation est abordé dans la suite de cette section.

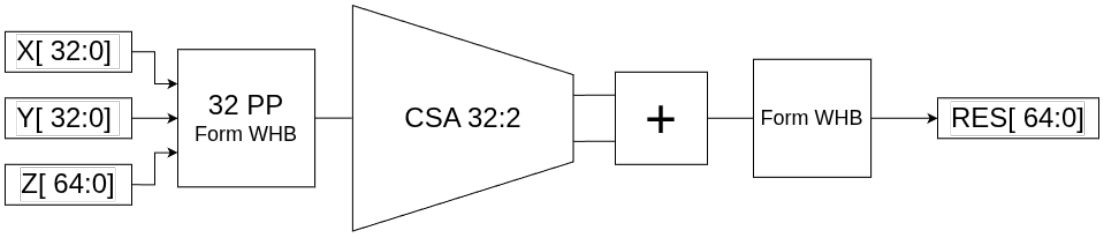


Figure 8: Multiplieur Vectoriel 32 bits

3.2.2 Formattage et découpage des différentes opérandes

Afin de pouvoir réutiliser l'arbre de Wallace du Multiplieur Scalaire, il est nécessaire de scinder les opérandes et d'étendre la taille des registres à 128 bits. L'objectif de ce découpage est d'isoler les différents blocs de données pour prévenir toute collision ou interférence entre les formats traités (Word, Half, Byte).

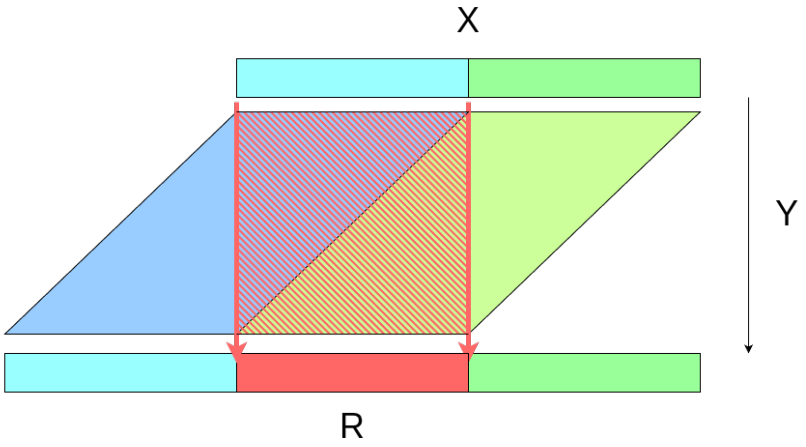


Figure 9: Chevauchement entre deux vecteurs au sein de l'opérande X

3.2.2.1 Découpage de l'opérande X

Le partitionnement de l'opérande X varie selon son format et la nature de l'opération. Dans le cas d'une opération **non signée**, les données sont réparties au sein d'un registre étendu à 128 bits de la manière suivante :

- **Format Word** : Le mot complet de 32 bits est conservé dans les bits de poids faible du registre étendu (bits 0 à 31).

- **Format Half** : L'opérande est scindé en deux demi-mots (*halfwords*). Le premier (H_0) est placé sur les bits de poids faible (0 à 15). Le second (H_1) est décalé et placé sur les bits 48 à 63. Cet espacement est stratégique : il anticipe le décalage de H_1 nécessaire lors de la génération et de l'accumulation des produits partiels, évitant ainsi toute collision.
- **Format Byte** : L'opérande est divisé en quatre octets (*bytes*). B_0 occupe les bits 0 à 7, B_1 les bits 24 à 31, B_2 les bits 48 à 55, et enfin B_3 les bits 72 à 79. Cette disposition répond à la même contrainte d'alignement que le mode *half* pour le calcul indépendant des produits partiels.

L'ensemble de ces configurations de placement est illustré à la Figure 10.

Pour les opérations impliquant des nombres signés, la logique de partitionnement s'appuie sur le même espacement de base que le type non signé, mais intègre une étape supplémentaire cruciale : l'extension de signe.

La particularité de ce mode réside dans le volume de cette extension : les bits de signe ajoutés occupent deux fois la place de la donnée d'origine. Ainsi, la taille totale allouée à chaque opérande est triplée (x3) au sein du registre étendu :

- **Format Word** : Le mot initial de 32 bits reçoit une extension de signe de 64 bits, occupant ainsi un espace total de 96 bits.
- **Format Half** : Chaque demi-mot de 16 bits (H_0, H_1) est complété par 32 bits de signe, portant sa taille totale à 48 bits avant d'être positionné.
- **Format Byte** : Chaque octet de 8 bits (B_0 à B_3) reçoit 16 bits d'extension de signe, occupant ainsi une empreinte totale de 24 bits.

Ce triplement de la taille garantit que les produits partiels, en particulier les valeurs négatives, conserveront une valeur mathématique rigoureusement exacte (préservation du complément à deux) lors de leur accumulation dans l'arbre de Wallace, en évitant toute corruption par les zéros des bits supérieurs. Et elle est illustré à la Figure 11.

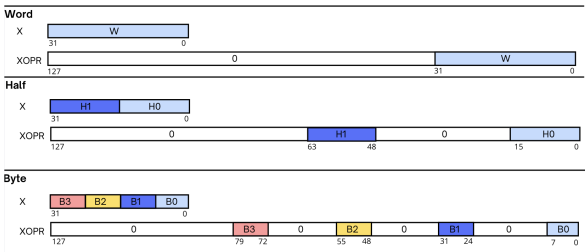


Figure 10: Découpage de X en non signé

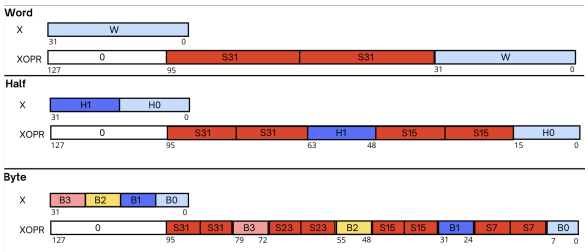


Figure 11: Extension de signe de l'opérande X

3.2.2.2 Découpage de l'opérande Z

L'opérande d'entrée Z, d'une largeur totale de 64 bits et réparti sur les deux registres architecturaux `High` et `Low`, nécessite un traitement spécifique avant son intégration dans l'arbre de calcul. Bien que la méthode de partitionnement reste similaire pour chaque format, seul le placement final des sous-opérandes diffère. Si l'on prend l'exemple du format *Half*, les registres `Low` et `High` sont d'abord scindés afin d'isoler indépendamment les parties basses et hautes des demi-mots H_0 et H_1. Ces données extraites alimentent ensuite un registre intermédiaire de 32 bits destiné à capturer H_0 ou H_1. C'est à cette étape qu'intervient une logique d'ajustement matériel par **ou exclusif (XOR)** : la valeur extraite subit une opération XOR avec un signal de contrôle. Ce signal est défini par l'équation logique $\text{Sub} \oplus (\text{SNG} \cdot S_{15/31})$, où Sub indique une soustraction, SNG une opération signée, et $S_{15/31}$ représente le bit de signe de l'opérande. Cette porte logique permet d'inverser conditionnellement la donnée pour gérer de manière unifiée et performante les soustractions et le traitement des nombres négatifs.

Ce mécanisme est illustré à la Figure 12.

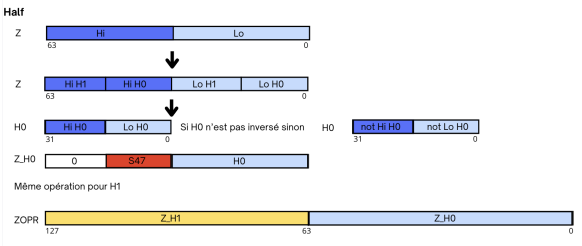


Figure 12: Exemple de découpage de l'opérande Z (Format Half)

À l'issue de ce conditionnement, le placement final des opérandes s'effectue de la manière suivante :

- **Format Word :** L'opérande s'étend sur l'intégralité du registre.
- **Format Half :** Placement séquentiel de H_1 suivi de H_0.
- **Format Byte :** Placement séquentiel de B_3 puis de B_2 suivi de B_1 finissant par B_0.

3.2.2.3 Inversion de l'opérande Y

De manière analogue à l'étape précédente, l'opérande Y doit subir une inversion conditionnelle avant d'être introduit dans l'arbre de calcul afin de permettre la réalisation des opérations de soustraction. Pour expliciter ce comportement, le **format Half** est à nouveau utilisé comme support d'illustration. Ce bloc utilise le signal de soustraction (`Sub`) et du bit de signe correspondant (Y_{31} ou Y_{15}) pour faire l'opération

Ce mécanisme est illustré à la Figure 13.

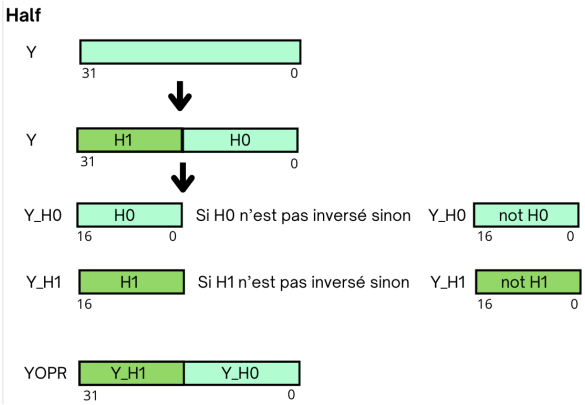


Figure 13: Mécanisme d'inversion de l'opérande Y (Format Half)

Selon le format sélectionné, l'inversion conditionnelle s'ajuste de la manière suivante :

- **Format Word :** L'inversion par complément à un est appliquée sur l'intégralité du registre, dépendante du bit de signe général Y_{31}.
- **Format Half :** Traitement séquentiel et indépendant où l'inversion s'effectue sur H_1 puis sur H_0, pilotée par le bit de signe Y_{15}.
- **Format Byte :** L'inversion est répartie de manière autonome sur les quatre octets en suivant la séquence B_3 puis B_2 suivi de B_1 et par B_0.

3.2.2.4 Découpage du registre de la somme résultante

À l'issue du calcul au sein de l'arbre de Wallace, le produit brut est stocké dans un registre de somme de 128 bits (`Sum`). L'étape finale consiste à extraire et réorganiser ces données pour les formater dans le registre de résultat final de 64 bits (`Res`). Cette extraction repose sur une logique de séparation des parties basses et hautes :

- **Format Word :** Le produit occupant les bits de poids faible, les 64 premiers bits (`[63:0]`) sont transférés directement.
- **Format Half :** Les parties basses (16 bits) des deux produits sont concaténées dans la moitié inférieure (`Res[31:0]`), tandis que les parties hautes (16 bits) sont regroupées dans la moitié supérieure (`Res[63:32]`).
- **Format Byte :** Selon la même logique d'entrelacement, les quatre parties basses de 8 bits sont juxtaposées dans `Res[31:0]`, et les quatre parties hautes dans `Res[63:32]`.

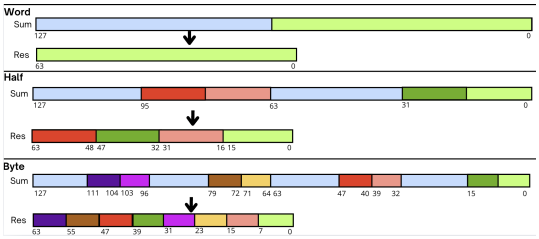


Figure 14: Extraction du résultat final depuis le registre de somme

Par ailleurs, un mécanisme d'inversion finale est opéré juste avant le placement dans le registre `Res`, faisant écho à la logique appliquée sur l'opérande Z. Si l'instruction exécutée est une soustraction, que le traitement est de type signé et que le format est négatif (bit de signe actif), une inversion bit à bit est alors appliquée sur les parties correspondantes du résultat.

3.3 Comparaison des Multiplieurs sur FPGA

FPGA utilisé: Zybo Z7-20
Contrainte d'horloge: $H = 10$ ns
Formule délai: $T = H - WNS$[1]
Formule fréquence: $f = \frac{1}{T}$

Multiplieur	T (ns)	f (MHz)	LUT
Scalaire	12,917	77,41	3812
Vectoriel	24,281	41,184	5692

Table 1: Comparaison des performances des multiplieurs

Nous pouvons observer que le Multiplieur Vectoriel est 1,88 fois plus lent et prend 1,49 fois plus de surface que le Multiplieur Scalaire. Ce qui est mieux en terme de place que 2,1 qui est le coût théorique de ce qu'aurait coûté en surface de rajouter deux multiplieurs: un pour le mode Half et un autre pour le mode Byte.
Une amélioration possible serait d'utiliser une architecture en pipeline pour le Multiplieur Vectoriel afin de réduire le chemin critique et augmenter sa fréquence.

[1] Le WNS (Worst Negative Slack) désigne la marge temporelle la plus défavorable du circuit, mesurant l'écart exact entre la contrainte d'horloge imposée et le temps de propagation réel sur le chemin critique.

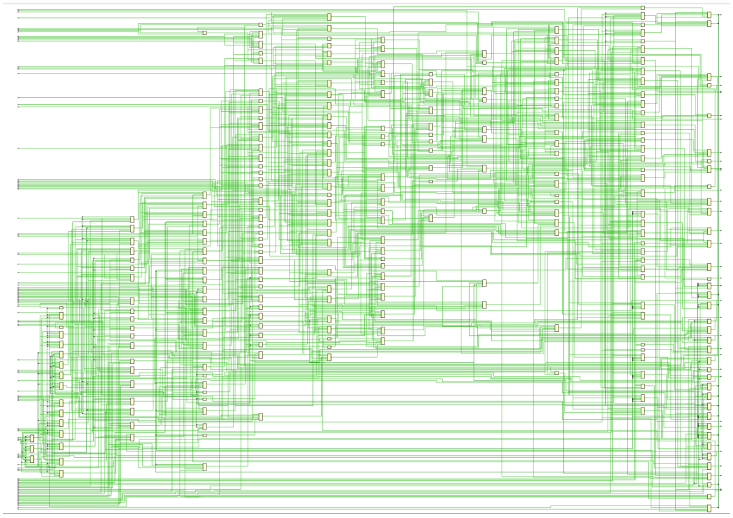


Figure 15: Implémentation du Multiplieur Vectoriel 32 bits sur FPGA

4 Procédure de recette

La validation de nos opérateurs repose sur une confrontation systématique entre notre conception et un modèle de référence. L'objectif est de vérifier l'intégrité de l'architecture réalisée à l'aide de fichiers de patterns.

4.1 Tests pour le Shifter et le Multiplieur

Pour le Multiplieur et pour chaque type d'opération du Shifter (*sll*, *srl*, *sra*, rotations droite et gauche), nous mettons en place une simulation de **10 millions de patterns**.

La procédure suit les étapes suivantes pour chaque opérateur:

- **Génération (Code C) :** Un programme C est utilisé pour construire le fichier de patterns. Pour chacune des opérations, il génère une série de 10 millions de tests aléatoires. Chaque ligne du fichier contient ainsi les opérandes et le résultat de référence calculé par le programme.
- **Description Matérielle (Code VHDL) :** Le Shifter et le Multiplieur sont intégralement décrits en VHDL, et implémentent leurs architectures respectives.
- **Simulation (Simulad) :** Un programme de simulation qui lit le fichier généré par le code C et injecte les vecteurs de données dans l'opérateur. Les résultats produits par le code VHDL sont ensuite comparés aux résultats de référence.

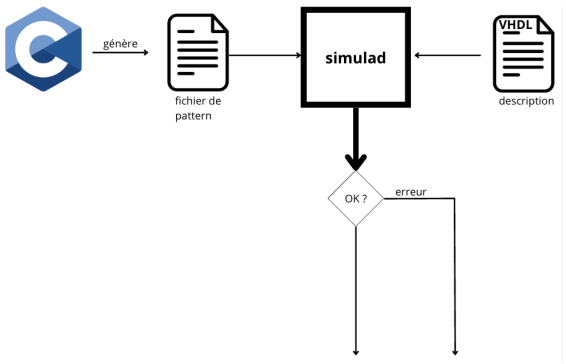


Figure 16: Détails de la procédure de recette et tests de validation

Bibliographie: *CMOS VLSI Design A Circuits and Systems Perspective(Fourth Edition), Neil Weste, David Harris*